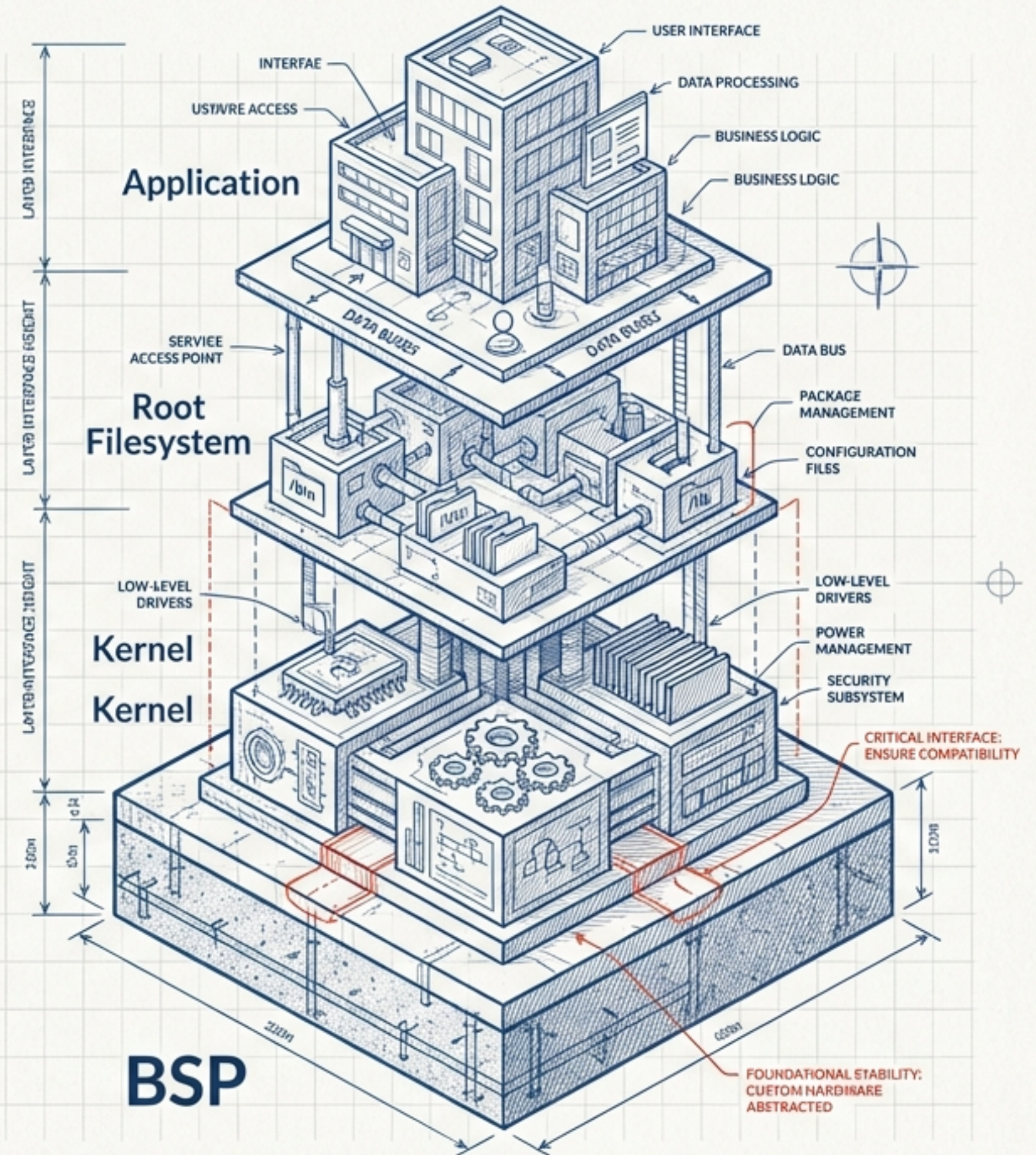


Architecting a Truly Good Board Support Package

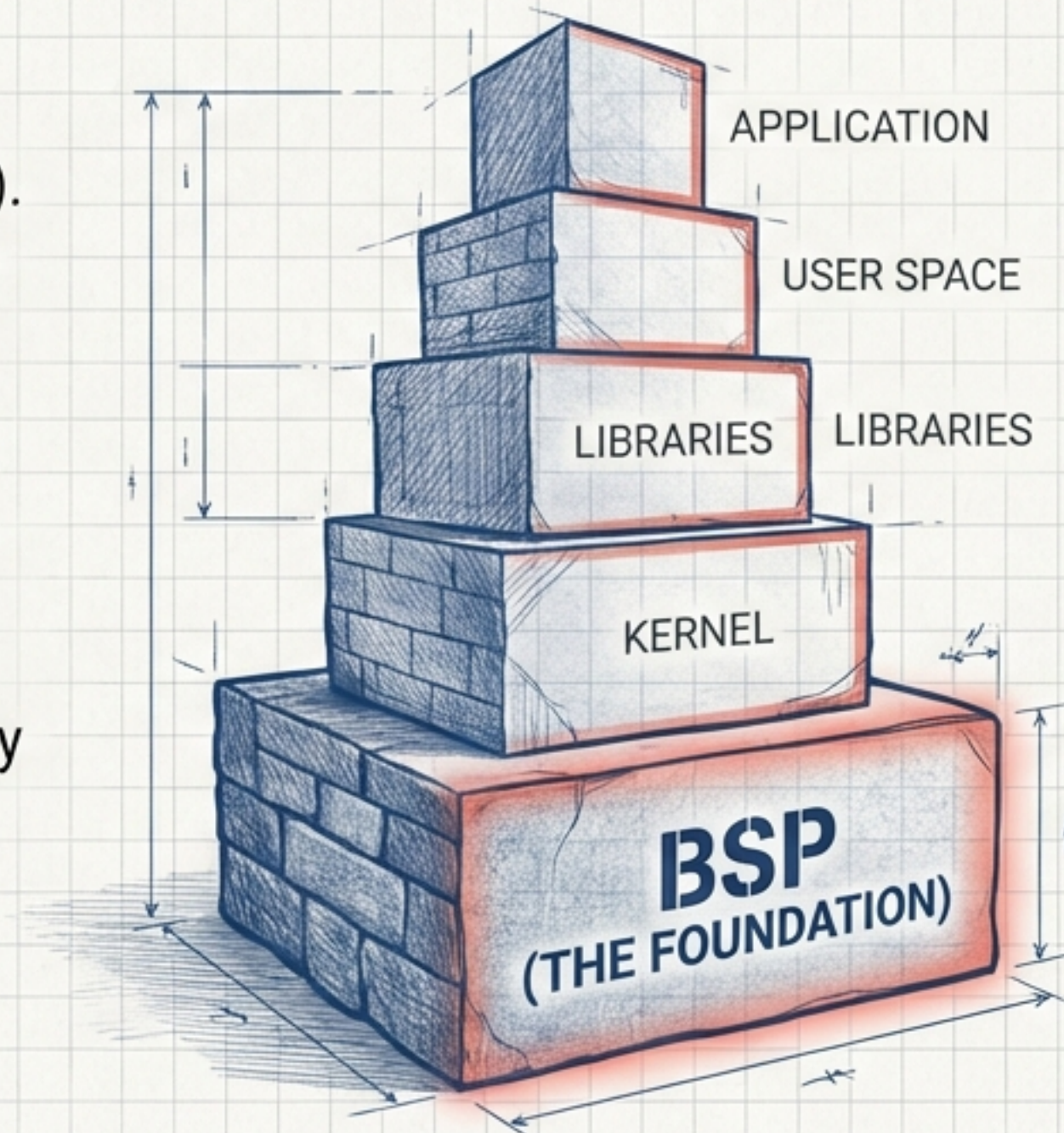
A Practical Guide to Building a Maintainable Foundation with the Yocto Project.



Your BSP is the foundation of your entire software stack.

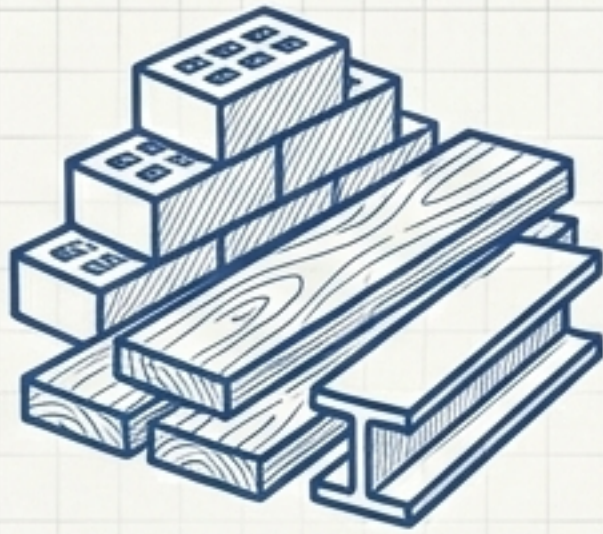
Every Yocto Project needs a Board Support Package (BSP). It defines the hardware and is the first layer in the stack you build upon.

Whether you receive it from a board vendor or write it for your own custom hardware, its quality dictates the stability and maintainability of your entire system.



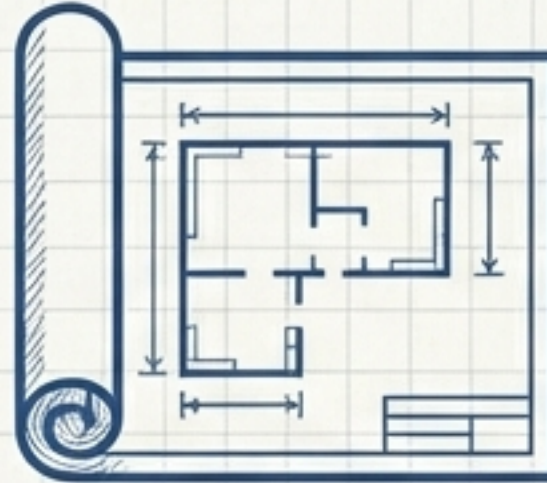
The BSP is, I believe, the most important part of the whole thing.

Understanding the Yocto Project's Toolkit



1. Recipes (`.bb` files): The raw materials.

Individual files containing instructions on how to fetch, configure, and compile a piece of software (e.g., a library, an application).



2. Layers (`.meta-\*`): The blueprints.

Collections of related recipes, organised into a modular directory structure. You combine layers to build your system.

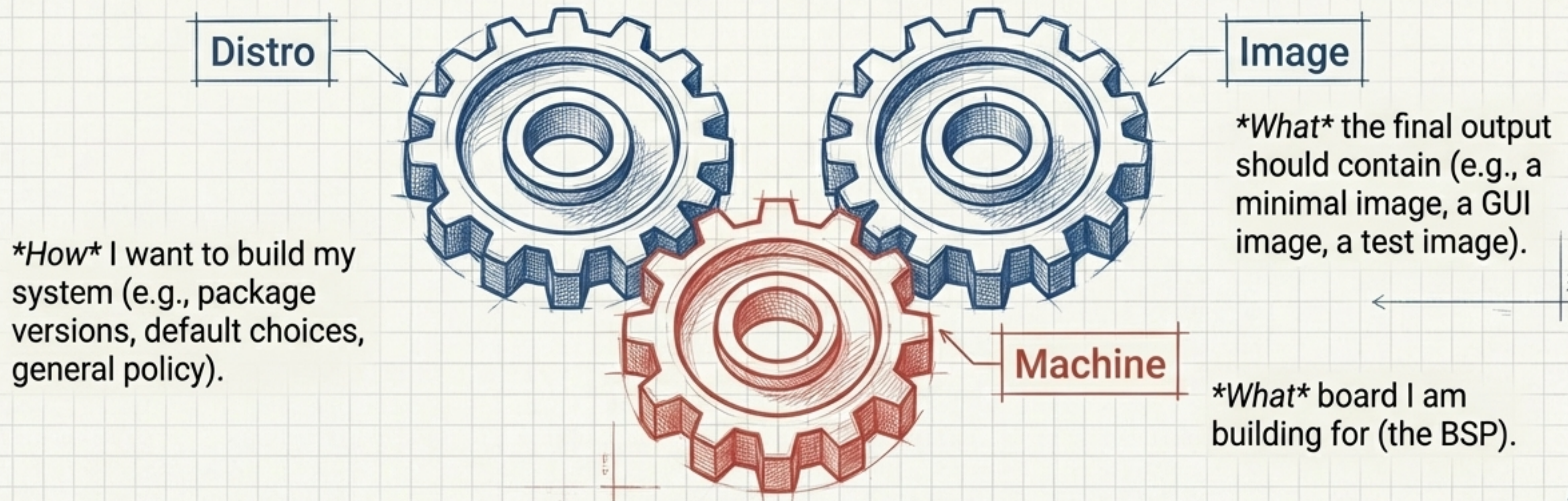


3. BitBake: The build engine.

The powerful task scheduler that parses the recipes and layers, resolves dependencies, and executes the tasks to build your final image.

The “Holy Trinity” of Yocto: The Power of Separation

Yocto's power comes from separating policy (Distro), hardware (Machine), and deliverables (Image). A well-designed system allows you to swap any one of these components without breaking the others.



Your BSP should define the **Machine** and nothing else.

The First Golden Rule of BSP Architecture: Keep It Simple.

The most common problem with BSPs is including too much. A BSP layer should only contain hardware-specific configuration. Anything generic belongs in a different layer.

What Belongs in a BSP (✓)

- Machine configuration file (.conf)
- Bootloader configuration (e.g., U-Boot)
- Kernel version and configuration
- Essential firmware binaries
- Hardware-specific components (e.g., GStreamer plugins for a specific video controller)

What Does **NOT** Belong (✗)

- Generic applications
- Corporate branding or splash screens
- Demo software
- Distro-level policy choices

Best Practice: If you are building a BSP, go see what meta-raspberrypi has done. It is a very good template.

Laying the Groundwork: Creating Your Machine Layer

1. Create the layer directory structure

```
# Creates the standard directory layout for your layer  
bitbake-layers create-layer meta-nova
```

2. Add the layer to your build

```
# Adds your new layer to conf/bblayers.conf  
bitbake-layers add-layer meta-nova
```

3. Define your machine configuration

Create the machine config file. The name of this file defines the MACHINE name.



```
meta-nova/conf/machine/nova.conf
```

***Pro Tip:** Don't forget to fill out the 'README' and 'LICENSE' files generated in your layer. An MIT license is common for layer metadata.*

Specifying the Bootloader and Kernel

Bootloader (U-Boot)

Add U-Boot as a build dependency in your nova.conf. It is built but not included in the root filesystem.

```
EXTRA_IMAGE_DEPENDS += "u-boot"
```

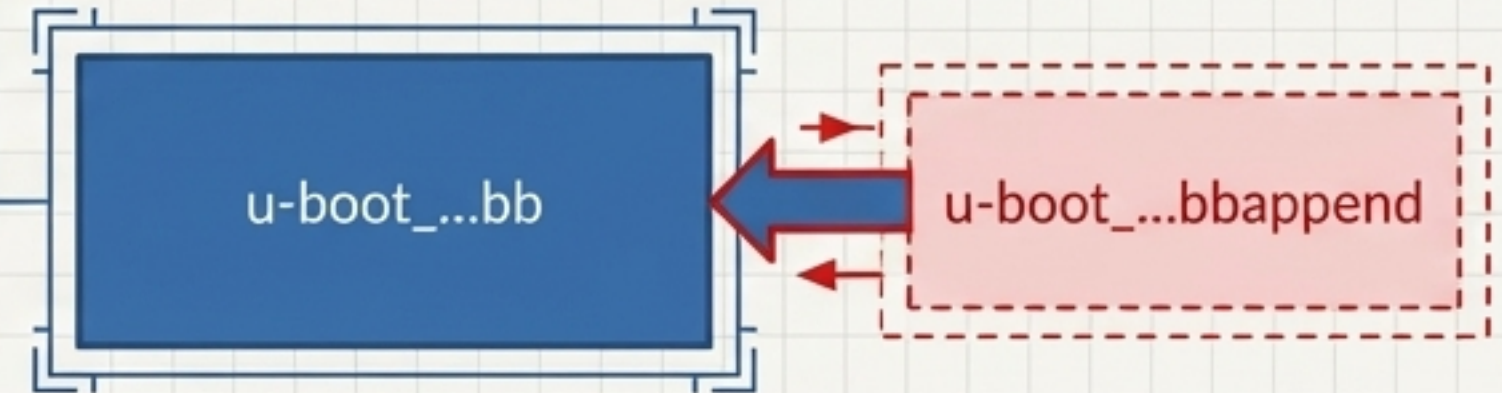
Kernel

Specify which kernel package should fulfill the role of virtual/kernel. Here, we use a custom kernel recipe called **linux-nova**.

```
PREFERRED_PROVIDER_virtual/kernel = "linux-nova"  
PREFERRED_VERSION_linux-nova = "5.10"
```

Customisation with bbappend

To configure U-Boot or the kernel for your specific board, you don't edit the original recipe. Instead, you create a .bbappend file in your layer. This file is virtually appended to the original recipe at parse time.



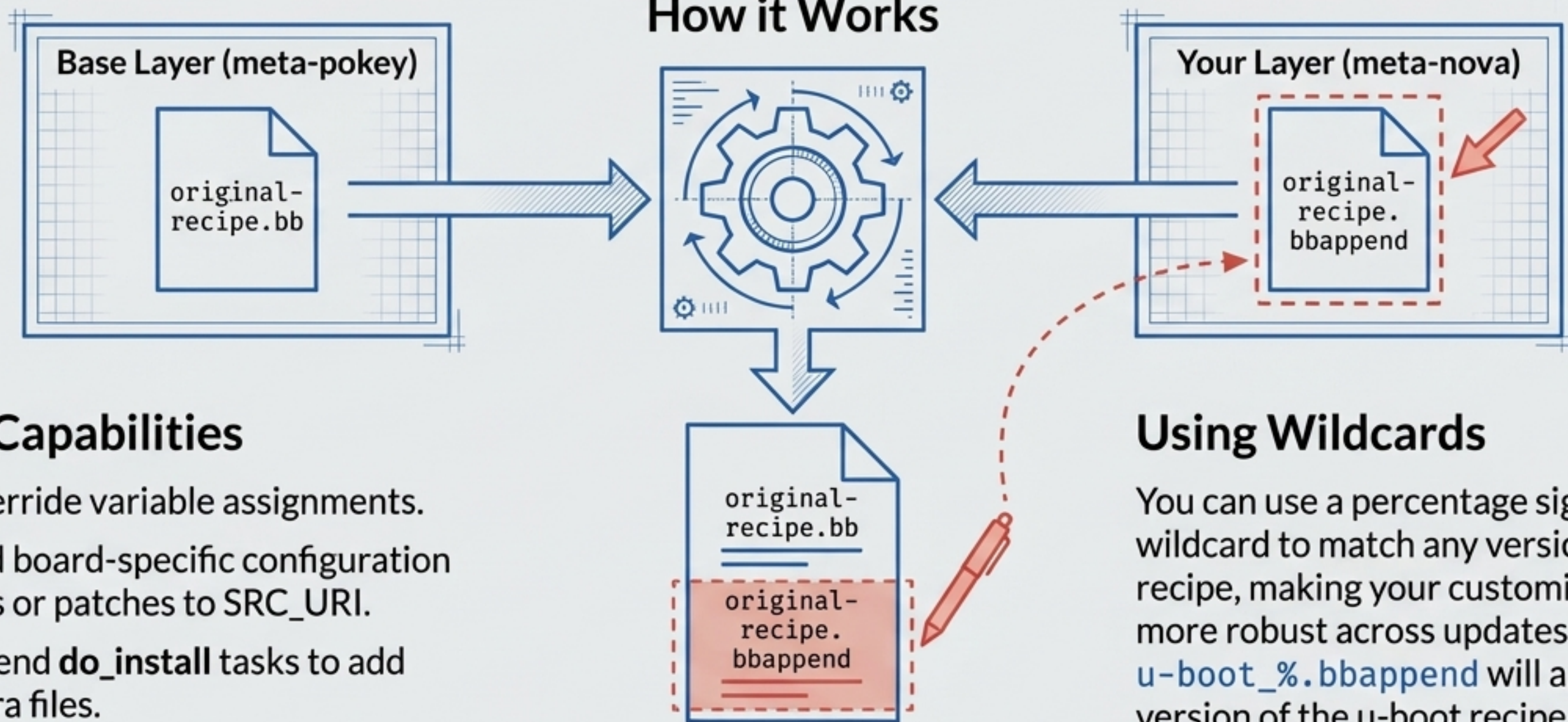
`meta-nova/recipes-bsp/u-boot/u-boot_%.bbappend`



Under the Hood: The Power of bbappend Files

.bbappend files are the cornerstone of Yocto's layering model. They allow you to modify or add to an existing recipe from within your own layer, without touching the original source.

How it Works



Key Capabilities

- ⊕ Override variable assignments.
- ⊕ Add board-specific configuration files or patches to `SRC_URI`.
- ⊕ Extend **do_install** tasks to add extra files.

Using Wildcards

You can use a percentage sign (%) as a wildcard to match any version of a recipe, making your customisations more robust across updates.

`u-boot_%.bbappend` will apply to any version of the u-boot recipe.

A Quick Guide to BitBake Assignment Operators

The operator you choose for variable assignment has a significant impact. Using the wrong one can make your BSP difficult for others to customise.

Operator	Name	Behaviour	Use Case in BSPs
<code>:=</code>	Immediate	Hard sets the variable's value at the time of parsing. Final and cannot be changed later.	Use for values that MUST NOT be changed.
<code>?=</code>	Weak	Assigns the value only if the variable is not already set.	Recommended default. Allows users to override in higher-priority layers.
<code>??=</code>	Default	Assigns the value at the very end of parsing if it was never set by <code>:=</code> or <code>?=</code> . The weakest assignment.	Useful for setting a sensible fallback.

Key Rule: When writing a BSP layer, prefer using `?=` for assignments. This allows a user to **easily override** your settings in a higher-level layer or their `local.conf`.



Managing the Device Tree and Firmware



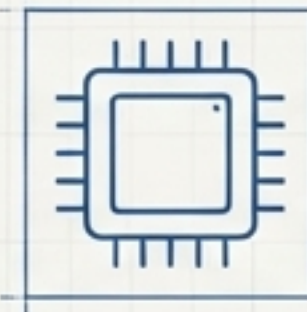
Device Tree (`.dts`)

The Device Tree describes the specific hardware on your board to the Linux kernel. You will almost always need to provide a custom one.

- ▶ Place your source file (e.g., `nova.dts`) within your kernel recipe's path.

Tell the kernel build system which device tree binaries (`.dtb`) to compile using `KERNEL_DEVICETREE` in your kernel's `.bbappend`.

```
KERNEL_DEVICETREE = "nova.dtb"
```



Firmware

Firmware files are binary blobs required for peripherals like Wi-Fi or Bluetooth chips.

Package the firmware in its own recipe.

- ▶ To ensure the firmware package is built and included in the final image, add it to `MACHINE_EXTRA_RRECOMMENDS` in your `nova.conf`.

```
# This ensures the firmware for the Wi-Fi  
chip is installed  
MACHINE_EXTRA_RRECOMMENDS += "linux-  
firmware-rpi-sdio"
```


Assembling the Final Image for Flashing

Defining Image Formats

Use `IMAGE_FSTYPES` in your `machine.conf` to specify which types of images to generate.

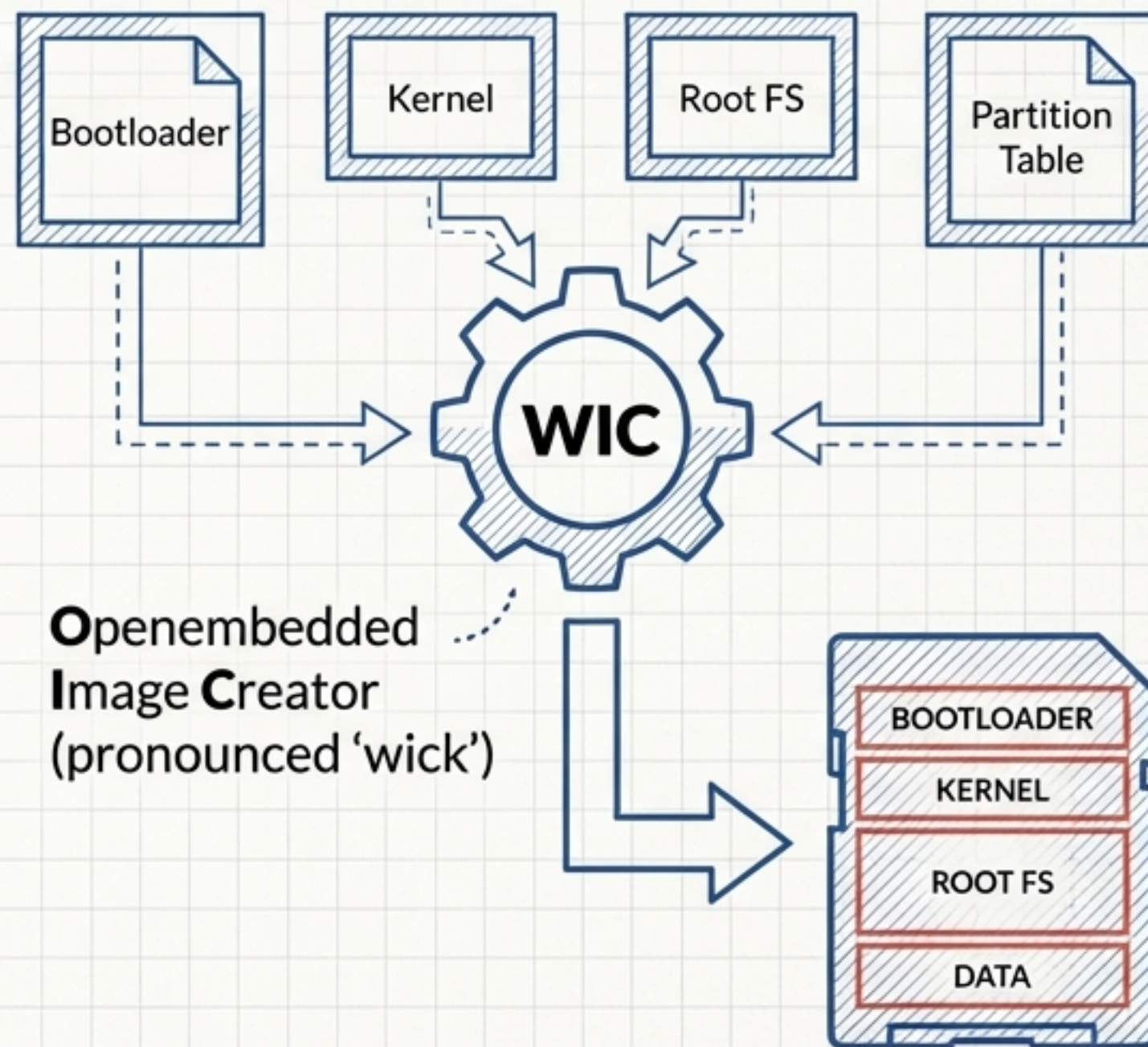
```
# Generates a rootfs tarball and a bootable .wic image  
IMAGE_FSTYPES = "tar.bz2 wic"
```

Creating Partitioned Images with WIC

Reference your WIC kickstart file (`.wks`) in your `machine.conf`:

```
# Tells WIC to use the beaglebone layout for this image  
WKS_FILE = "beaglebone-yocto.wks"
```

What is WIC?

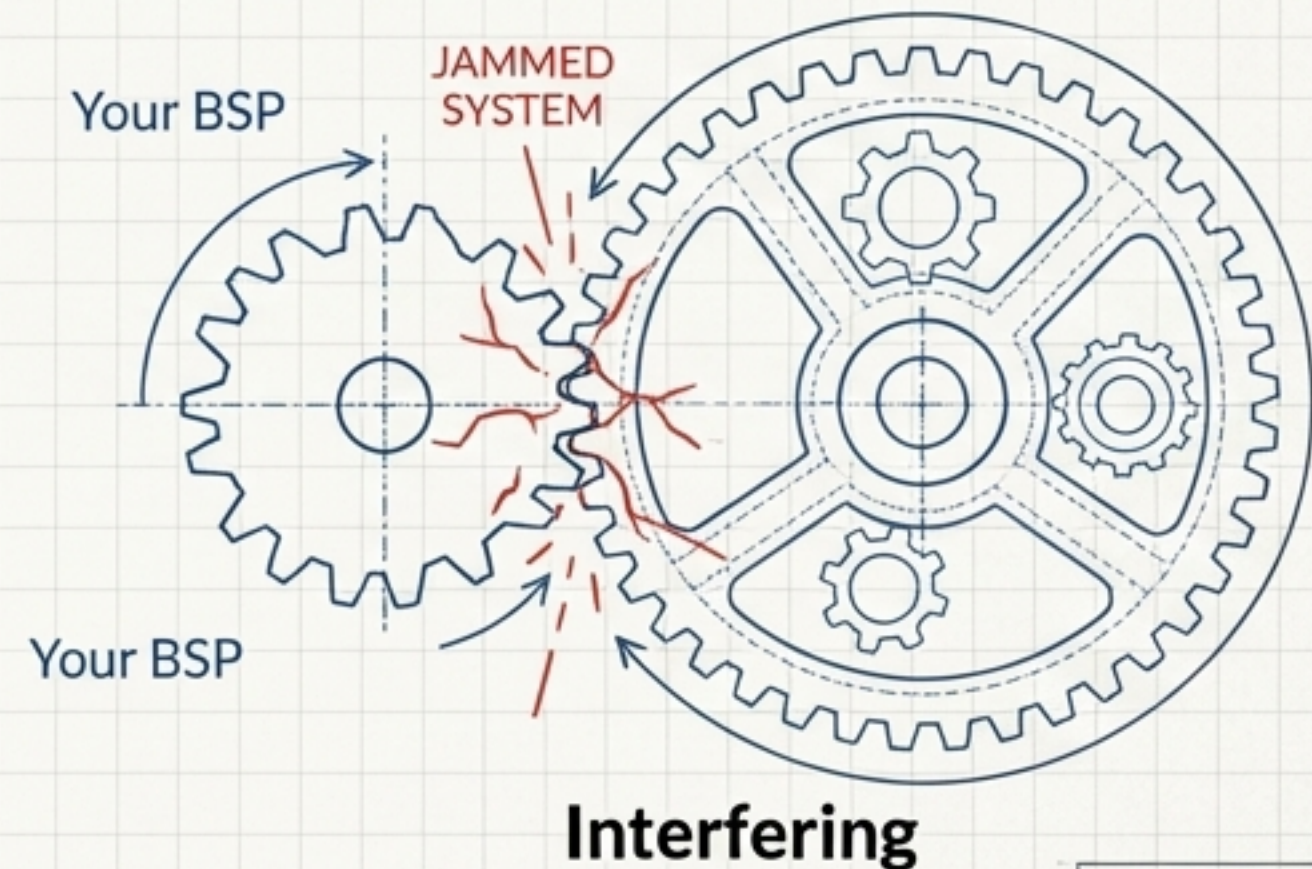
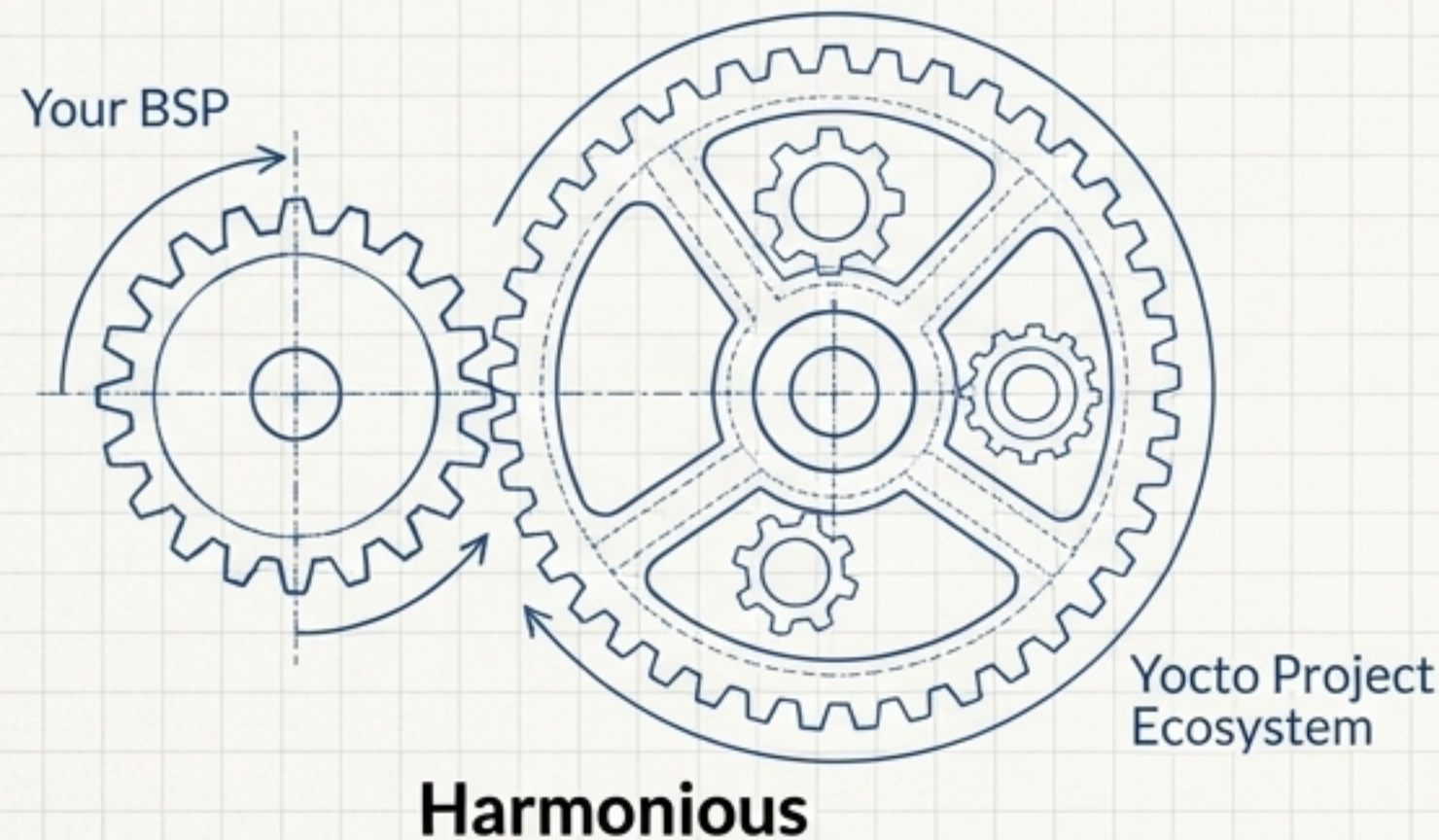


The Second Golden Rule: Don't Break Things

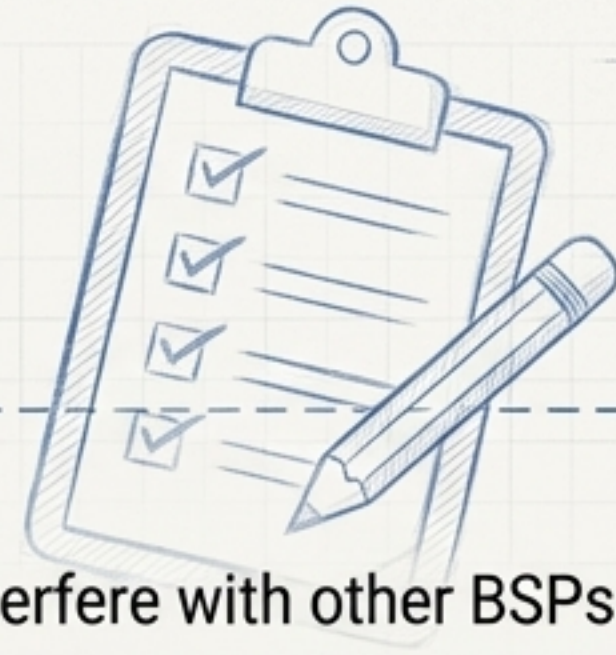
A well-written BSP works flawlessly for its target machine. A great BSP does so without interfering with any other machine configuration in the project. Many BSPs are written with the flawed assumption that they will be the only one ever used in a build.

The Litmus Test

"If you add your BSP layer to bbblayers.conf but select a different MACHINE to build, the build should still complete without errors. If it fails, your BSP layer is not friendly."



A Checklist for Quality and Sanity



Overly Aggressive `bbappend` Files

Problem: A `bbappend` for a common component like the kernel can interfere with other BSPs.

Solution: Make your `bbappend` files conditional on your machine (`COMPATIBLE_MACHINE = "nova"`). Test builds for other machines.

Problem: Hidden Layer Dependencies. The build fails with 'Could not include required file...' because a required layer is missing.

Solution: Explicitly declare your layer's dependencies in its `meta-nova/conf/layer.conf` file.

```
# This layer depends on the core and freescale layers  
LAYERDEPENDS_meta-nova = "core meta-freescale"
```

Problem: Unchecked Errors. The layer contains common syntax errors, incorrect paths, or other issues.

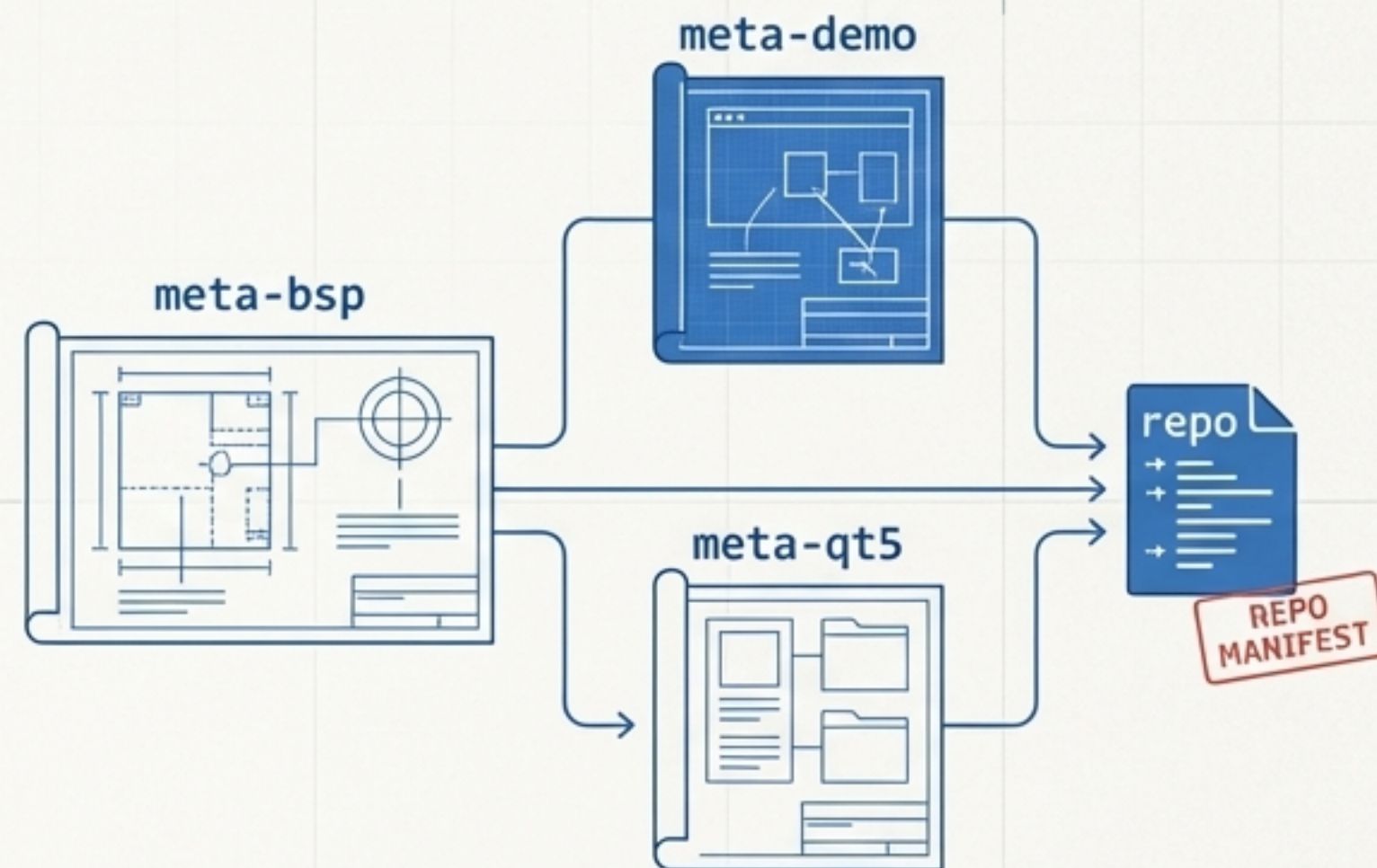
Solution: Regularly run the `yocto-check-layer` script on your layer. It performs various heuristic tests and will save you hours of debugging.



Your BSP is an Engineering Tool, Not a Marketing Showcase.

The Problem: Bloated BSPs

BSPs from vendors are often bloated with demos, company splash screens, and other non-essential software. This is done to “showcase” the product but creates unnecessary work for the developer who has to remove it all.



The Solution: Separate Your Demos

Keep your BSP layer lean and focused *only* on making the hardware work. If you want to provide demos or example applications, put them in a *separate meta-*-demo* layer.

The Rewards of a Well-Architected BSP

Why Getting It Right Matters:



Reduced Development Costs: A clean, modular BSP is easier to maintain, update, and debug.



Easier Product Integration: Customers can integrate your board into their existing Yocto builds without conflict.



Help from the Community: A standards-compliant BSP makes it easier for Yocto developers and the community to help you.



Happier Users: Developers can get to work on their applications faster, without having to clean up your BSP first.



Once your BSP is validated and polished, publish it to the ****OpenEmbedded Layer Index**** (layers.openembedded.org). This makes your hardware accessible to the entire community and is the hallmark of a truly great BSP.